

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Отчёт по лабораторной работе № 4
«Проблема бинаризации признаков»

Выполнил работу
Заборщиков Артём
Академическая группа №J3110
Принято
Ментор, Владислав Вершинин

Санкт-Петербург

2024

Введение	2
Подсчёт по памяти	2
Подсчёт по времени	6
График зависимости времени от числа элементов	7

Введение

Задача - решить пр полную задачу по перебора признаков. Для решения задачи были изучены методы оптимизации итогового кода, многопоточные вычисления.

Подсчёт по памяти

```
#include <iostream>
#include <cmath>
#include <chrono>
#include <fstream>
#include <string>
#include <sstream>
#include <vector>
#include <mutex>
#include <thread>
#include <atomic>
#include <set>
#include <chrono>

#define ARMA_WARN_LEVEL 1
#define MLPACK_PRINT_INFO
#define MLPACK_PRINT_WARN
#include <mlpack.hpp>
#include "modelling.cpp"

std::mutex cout_mutex;

// Функция для бинаризации данных  $O(n*k)$  - создается новая таблица
таких же размеров, как и прошлая
arma::mat Binarize(const arma::mat& dataset, const
std::vector<double>& thresholds, int cols) {
    arma::mat binary_dataset = dataset;
    for (int j = 0; j < cols; ++j) {
        binary_dataset.row(j) =
arma::conv_to<arma::mat>::from(binary_dataset.row(j) > thresholds[j]);
    }
    return binary_dataset;
}

double feature_selection(const int& train_cols = 11, const int& offset
= 30000, bool print_best = false) {
    const char* path = "C:/Users/a1az1/Downloads/WineQT.csv";
    const int cols = 11;
    const int shift = cols - train_cols;
```

```

const int ntrasholds = 5;
double time_elapsed_global = 0;

arma::mat dataset;
// O(n*k) - строки*столбцы
if (!mlpack::data::Load(path, dataset)) {
    throw std::runtime_error("Could not read *.csv!");
}

if (dataset.is_empty()) {
    throw std::runtime_error("Loaded dataset is empty!");
}

size_t num_rows = dataset.n_cols;
size_t num_rows_percent = static_cast<size_t>(std::ceil(num_rows *
1));
// O(n*k) дважды, создаём две новые таблицы
arma::mat selectedrows = dataset.cols(0, num_rows_percent - 1);
arma::mat subset = selectedrows.rows(shift, cols + 1); // + 2
(индексы + y) - 1 (все таки индекс, а не количество) = + 1

std::cout << "Dataset shape: " << subset.n_rows << "x" <<
subset.n_cols << std::endl;

// Предварительное вычисление пороговых значений, O(n*k)
std::vector<std::vector<double>> thresholds(train_cols,
std::vector<double>(ntrasholds));
for (int j = 0; j < train_cols; ++j) {
    double min_val = subset.row(j).min();
    double max_val = subset.row(j).max();
    double range = (max_val - min_val) / (ntrasholds + 1);
    for (int s = 1; s <= ntrasholds; ++s) {
        thresholds[j][s - 1] = min_val + range * s;
    }
}

double best_score = INFINITY;
unsigned int best_idx;
auto t_start = std::chrono::high_resolution_clock::now();

```

```

    auto iters = static_cast<long long>(std::pow(ntrasholds,
train_cols));
    std::atomic<long long> iteration_counter = 0;

    const int num_threads = std::thread::hardware_concurrency();
    std::vector<std::thread> threads;

    auto worker = [&](int thread_id) {
        arma::mat post_dataset; //  $O(n*k)$  - таблица
        double local_best_score = INFINITY;
        int local_best_idx = -1;

        for (long long i = thread_id; i < iters; i += num_threads) {
            // Вычисляем текущее сочетание порогов
            std::vector<double> current_thresholds(train_cols); //
             $O(1)$ , размер фиксирован и не слишком велик, нотацию можно зажать
            между константами в случае адекватных значений

            long long idx = i;

            for (int j = 0; j < train_cols; ++j) {
                int
                step = idx % ntrasholds;
                current_thresholds[j] = thresholds[j][step];
                idx /= ntrasholds;
            }

            // Бинаризуем данные,  $O(n*k)$ 
            post_dataset = Binarize(subset, current_thresholds,
train_cols);

            // Оцениваем модель
            double score = evaluate_dataset(post_dataset, 11 - shift,
12 - shift);

            // Обновляем локальный лучший результат
            if (score < local_best_score) {
                local_best_score = score;
                local_best_idx = i;
            }

            // Обновляем общий лучший результат (без гонок данных)
            {
                std::lock_guard<std::mutex> lock(cout_mutex);

```

```

        if (local_best_score < best_score) {
            best_score = local_best_score;
            best_idx = local_best_idx;
            if (print_best) std::cout << "New best score! IDX:
" << best_idx << ", RMSE: " << best_score << '\n';
        }
    }

    ++iteration_counter;

    // Периодический вывод прогресса
    if (iteration_counter % offset == 0) {
        auto elapsed_seconds =
std::chrono::high_resolution_clock::now() - t_start;
        double progress =
static_cast<double>(iteration_counter) / iters;
        double time_left = (elapsed_seconds.count() /
progress) * (1 - progress);

        {
            std::lock_guard<std::mutex> lock(cout_mutex);
            std::cout << "Thread ID: " << thread_id
                << " (" << progress * 100 << "%) Iteration: "
<< iteration_counter
                << " RMSE: " << score
                << " Best RMSE: " << best_score
                << " Elapsed Time: " <<
elapsed_seconds.count() / 1e9
                << " s, Time Left: " << time_left / 1e9 << "
s\n";
        }
    }
}

};

for (int t = 0; t < num_threads; ++t) {
    threads.emplace_back(worker, t);
}

for (auto& thread : threads) {
    thread.join();
}

```

```

    auto elapsed_seconds = (std::chrono::high_resolution_clock::now()
- t_start).count() / 1e9;

    std::cout << "Best trasholds: ";
    for (int j = 0; j < train_cols; ++j) {
        int step = best_idx % ntrasholds;
        std::cout << step;
        best_idx /= ntrasholds;
    }
    std::cout << '\n' << "Best RMSE: " << best_score << '\n'
        << "Elapsed Time: " << elapsed_seconds << '\n'
        << "Cols used: " << train_cols << '\n';

    return best_score;
}

void run_tests() {
    std::cout << "Best RMSE: " << feature_selection();
}

int main() {
    for (int i = 1; i < 11 + 1; i++) feature_selection(i, 1000000000,
false);
    return 0;
}

```

Просуммировав нотации можем получить $O(n*k)$

Подсчёт по времени

Рассмотрим код в порядке выполнения

```
for (int i = 1; i < 11 + 1; i++) feature_selection(i, 1000000000,
false);
```

Вызов 11 функций вычисления лучших порогов

```

    std::vector<std::vector<double>> thresholds(train_cols,
std::vector<double>(ntrasholds));
    for (int j = 0; j < train_cols; ++j) {
        double min_val = subset.row(j).min();
        double max_val = subset.row(j).max();
    }

```

```

double range = (max_val - min_val) / (ntrasholds + 1);
for (int s = 1; s <= ntrasholds; ++s) {
    thresholds[j][s - 1] = min_val + range * s;
}
}

```

Предварительное вычисление значения порогов - $O(n*k)$

```
auto worker = [&](int thread_id)
```

Запуск параллельного вычисления t итераций

$t = n_trasholds^{train_cols}$

```

for (int j = 0; j < train_cols; ++j) {
    step = idx % ntrasholds;
    current_thresholds[j] = thresholds[j][step];
    idx /= ntrasholds;
}

```

Вычисление порога $O(1)$ - в случае адекватных значений $train_cols$ можно довольно строго зажать константами

```
post_dataset = Binarize(subset, current_thresholds, train_cols);
```

Бинаризация - $O(n*k)$ - проходим всю таблицу

```
double score = evaluate_dataset(post_dataset, 11 - shift, 12 - shift);
```

O -нотацию применить по идее невозможно так как мы не используем знания о модели, которую обучаем, однако можно принять её за $O(n*k)$ (в целом примрено так и есть, по исходному коду можно узнать что это линрег)

В итоге мы получаем 11 запусков функции, которая работает за $O(n*k) + k^n * O(n*k) = O(k^n)$

График зависимости времени от числа элементов

Значения незначительно округлены, для последнего случая время расчетное. Используется логарифмическая шкала для отображения на графике.

iter	1	2	3	4	5	6	7	8	9	10	11
t (с)	0.002	0.004	0.02	0.075	0.3	1.9	11.2	63	321	1643	8958
Темп	-	2.0	5.0	3.75	4.0	6.33	5.89	5.62	5.1	5.12	5.45

Темп роста времени в общем и целом соответствует нотации ($k = 5$)

